Avaliação de Protocolos de Comunicação em Sistemas IoT Médicos com Arquitetura Multi-Cloud

João Tejo[1], Thiago Bezerra [1], Eduardo Tavares[1]

[1]Centro de Informática,
Universidade Federal de Pernambuco, Recife, Brasil.

{jlvat,tvb,eagt}@cin.ufpe.br

Abstract. *Internet of Medical Things has expanded the use of connected devices for patient monitoring, but healthcare applications require not only low communication delay, but also availability, fault tolerance, and data integrity. This paper presents an experimental evaluation of the performance and reliability of communication protocols applied to IoT systems in the medical context, considering realistic operation and failure scenarios. For this purpose, a distributed multi-cloud architecture was developed with active redundancy, data replication, and automatic failover between cloud providers, using embedded devices and MQTT-based data transmission. The analysis considers both performance and reliability metrics, including latency, jitter, availability, failover time, recovery time, packet loss, and packet duplication. The results aim to support the assessment of IoT protocols in critical healthcare applications by considering both normal operating conditions and system behavior under failures.*

Resumo. *A Internet das Coisas Médicas tem ampliado o uso de dispositivos conectados no monitoramento de pacientes, porém aplicações em saúde exigem não apenas baixo atraso na comunicação, mas também disponibilidade, tolerância a falhas e integridade dos dados. Este trabalho apresenta uma avaliação experimental do desempenho e da confiabilidade de protocolos de comunicação aplicados a sistemas IoT no contexto médico, considerando cenários realistas de operação e falha. Para isso, foi desenvolvida uma arquitetura distribuída baseada em multi-cloud, com redundância ativa, replicação de dados e failover automático entre provedores de nuvem, utilizando dispositivos embarcados e transmissão via MQTT. A análise considera métricas de desempenho e confiabilidade, incluindo latência, jitter, disponibilidade, tempo de failover, tempo de recuperação, perdas e duplicações de pacotes. Os resultados buscam apoiar a avaliação de protocolos IoT em aplicações médicas críticas, considerando tanto o comportamento em condições normais quanto a resposta do sistema diante de falhas.*

1. Introdução

A Internet das Coisas Médicas (IoMT) tem ampliado o uso de dispositivos conectados no monitoramento contínuo de pacientes, integrando sensores, dispositivos embarcados, redes de comunicação e serviços em nuvem. Em aplicações dessa natureza, a comunicação de dados precisa ocorrer com regularidade e dentro de limites aceitáveis de atraso, pois interrupções, perdas de mensagens ou degradações prolongadas podem comprometer a continuidade do monitoramento e o funcionamento do sistema [Rahmani et al. 2015].

Embora existam estudos sobre protocolos de comunicação para IoT, muitas avaliações ainda são conduzidas em condições controladas, com maior foco em métricas tradicionais de desempenho e menor atenção ao comportamento do sistema diante de falhas. Em aplicações médicas, essa análise é limitada, pois o funcionamento da solução depende não apenas da eficiência da comunicação, mas também da capacidade de manter o serviço disponível e recuperar a operação após eventos adversos [Chen and Kunz 2016].

Nesse contexto, este artigo apresenta uma avaliação experimental do desempenho e da confiabilidade de protocolos de comunicação aplicados a sistemas IoT no contexto médico, considerando cenários realistas de operação e falha. Para isso, foi desenvolvida uma arquitetura IoT distribuída baseada em multi-cloud, com redundância ativa, replicação de dados e failover automático entre provedores de nuvem. A solução utiliza dispositivos embarcados e transmissão via MQTT, permitindo observar o comportamento do sistema tanto em condições normais quanto em situações de instabilidade induzidas de forma controlada [Neto et al. 2019].

A análise considera métricas de desempenho e confiabilidade, incluindo latência, jitter, disponibilidade, tempo de failover, tempo de recuperação, perdas e duplicações de pacotes. Com isso, o trabalho busca examinar de forma conjunta o comportamento da comunicação e a resposta do sistema diante de falhas, oferecendo uma metodologia experimental reprodutível para avaliação de protocolos IoT em aplicações médicas com requisitos de continuidade de serviço.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta os trabalhos relacionados; a Seção 3 discute os conceitos teóricos utilizados; a Seção 4 descreve a metodologia experimental; a Seção 5 apresenta a arquitetura proposta; a Seção 6 discute os resultados obtidos; e a Seção 7 apresenta as conclusões e os trabalhos futuros.

2. Trabalhos Relacionados

Esta seção apresenta trabalhos recentes relacionados à comunicação, disponibilidade e confiabilidade em sistemas IoT aplicados à saúde.

Lakhan et al. [Lakhan et al. 2023] propõem um framework tolerante a falhas para IoT industrial em saúde baseado em *digital twin* e redes fog-cloud federadas, no qual tarefas computacionais são distribuídas entre nós de fog e múltiplos provedores de nuvem com mecanismo de failover automático. A avaliação considera latência como principal métrica, mas não adota injeção de falhas controlada nem inclui métricas como jitter, taxa de entrega de pacotes ou tempo de recuperação. Cao et al. [Cao et al. 2019] apresentam uma arquitetura multi-cloud escalável para IoT médica, em que dados clínicos são distribuídos e replicados entre provedores distintos para favorecer disponibilidade e escalabilidade de armazenamento. A análise considera tempo de resposta, mas não inclui failover automático no nível do dispositivo nem cenários com falhas induzidas para observar o comportamento do sistema sob interrupções.

Upadhyay et al. [Upadhyay et al. 2023] analisam desafios e limitações de sistemas IoT para saúde inteligente, comparando protocolos como MQTT, CoAP e HTTP quanto à adequação em cenários com dispositivos de recursos limitados. O estudo considera latência de transmissão, mas não aborda redundância entre provedores de nuvem nem mecanismos de tolerância a falhas na infraestrutura. Tareq e Khaleel [Wadullah Tareq and Ahmed Khaleel 2021] estudam a implementação do protocolo

MQTT em sistemas de saúde baseados em IoT, avaliando o desempenho da comunicação em termos de latência e taxa de entrega de pacotes em diferentes níveis de QoS. O foco permanece na comunicação, sem considerar arquiteturas distribuídas com replicação de dados ou comutação entre servidores.

Stergiou et al. [Stergiou et al. 2023] investigam a transmissão segura de grandes volumes de dados IoT em um cenário de *digital twin* para saúde, propondo um modelo de replicação de dados entre camadas de nuvem com ênfase em confidencialidade e integridade. A avaliação utiliza *throughput* como métrica de desempenho, mas não considera redundância ativa entre provedores nem mecanismos de failover. Sornlertlamvanich et al. [Sornlertlamvanich et al. 2025] propõem uma arquitetura de telemonitoramento remoto que integra dispositivos IoT, gateway doméstico e nuvem, com segurança ponta a ponta via HTTPS sobre QUIC. O protótipo utiliza Raspberry Pi 5 com sensores de ECG, SpO_2 e frequência cardíaca, e a avaliação experimental mede a latência de transmissão com e sem criptografia adicional, sem diferença relevante entre os cenários analisados.

A Tabela 1 apresenta um resumo comparativo dos trabalhos revisados. Os estudos analisados concentram-se em aspectos específicos, como latência, *throughput*, tempo de resposta, segurança, armazenamento distribuído ou comparação entre protocolos. Neste conjunto, a avaliação integrada de métricas de desempenho e confiabilidade em cenários com falhas controladas aparece com menor frequência, principalmente quando se considera failover automático, disponibilidade do serviço, jitter, taxa de entrega de pacotes e tempo de recuperação.

Diferentemente dos trabalhos analisados, a abordagem proposta neste artigo reúne, em um mesmo ambiente experimental, uso de MQTT na comunicação entre dispositivo e nuvem, arquitetura multi-cloud com redundância entre provedores, failover automático no dispositivo embarcado e replicação de dados entre instâncias. Além disso, a avaliação considera métricas de comunicação e indicadores relacionados à continuidade do serviço sob falhas, incluindo disponibilidade, jitter, taxa de entrega de pacotes, tempo de failover e tempo de recuperação.

Tabela 1. Resumo dos trabalhos relacionados.

Trabalho	Multi-Cloud	Failover	Replic.	MQTT	Métricas avaliadas
[Lakhan et al. 2023]	•	•	–	–	Latência
[Stergiou et al. 2023]	–	–	•	–	Throughput
[Upadhyay et al. 2023]	–	–	–	•	Latência
[Sornlertlamvanich et al. 2025]	•	–	–	•	Latência
[Cao et al. 2019]	•	–	•	–	Tempo de resposta
[Wadullah Tareq and Ahmed Khaleel 2021]	–	–	–	•	Latência, PDR
Este trabalho	•	•	•	•	Latência, jitter, disponibilidade, PDR, tempo de failover, tempo de recuperação

3. Referencial Teórico

Esta seção apresenta os conceitos fundamentais necessários para a compreensão da análise proposta.

3.1. Internet das Coisas Médicas

A Internet das Coisas (IoT) corresponde à integração de dispositivos físicos, sensores, atuadores e serviços computacionais conectados em rede, permitindo coleta, transmissão e processamento de dados em diferentes contextos. Na área da saúde, essa abordagem tem sido aplicada por meio da Internet das Coisas Médicas (IoMT), que reúne dispositivos e sistemas voltados ao monitoramento de pacientes, acompanhamento remoto e apoio a atividades assistenciais.

Em aplicações médicas, a IoMT permite o envio contínuo de sinais e eventos clínicos, favorecendo maior visibilidade sobre o estado do paciente e permitindo respostas mais rápidas em situações de alteração. No entanto, esses sistemas operam em contextos mais sensíveis, nos quais atrasos na comunicação, indisponibilidade de serviços ou inconsistências nos dados podem comprometer a continuidade do monitoramento. Por essa razão, aplicações IoMT exigem não apenas conectividade entre dispositivos, mas também regularidade na transmissão, disponibilidade do serviço e integridade das informações.

3.2. Protocolos de Comunicação em Sistemas IoT

A comunicação de dados é uma das bases do funcionamento de sistemas IoT, uma vez que os dados coletados pelos dispositivos precisam ser encaminhados de forma contínua para componentes de processamento, armazenamento e visualização. Nesse contexto, a escolha do protocolo e da tecnologia de comunicação influencia aspectos como atraso na entrega, estabilidade da transmissão, consumo de recursos e capacidade de operação em diferentes cenários.

Entre os protocolos utilizados em aplicações IoT, o MQTT tem recebido atenção por adotar o modelo publish/subscribe, no qual dispositivos publicam mensagens em tópicos e outros componentes as consomem de acordo com seu interesse. Esse modelo favorece desacoplamento entre produtores e consumidores de dados, facilitando a integração entre dispositivos e serviços distribuídos. Além disso, o MQTT oferece níveis de qualidade de serviço que podem ser ajustados conforme a necessidade da aplicação.

No nível de conectividade, diferentes tecnologias podem ser utilizadas conforme o ambiente e os requisitos da aplicação. Redes WiFi são frequentemente utilizadas em cenários internos e em situações que demandam maior taxa de transmissão. Já tecnologias como LoRaWAN têm sido consideradas em aplicações IoT por oferecerem maior alcance e menor consumo energético, embora apresentem limitações quanto à vazão e ao atraso. Dessa forma, a escolha da solução de comunicação deve considerar as características do ambiente e as exigências do sistema monitorado.

3.3. Métricas de Avaliação

A avaliação de sistemas IoT distribuídos requer métricas que permitam observar tanto o comportamento da comunicação quanto a capacidade do sistema de manter sua operação diante de falhas. Neste trabalho, são consideradas métricas de desempenho e confiabilidade, de modo a analisar o funcionamento da arquitetura em condições normais e sob falhas controladas.

A disponibilidade (A) é utilizada para representar a razão entre o tempo em que o sistema permaneceu operacional e o tempo total de execução, sendo expressa em porcen-

tagem. A latência de publicação (L) corresponde ao intervalo entre o envio de uma mensagem MQTT e a confirmação de recebimento pelo broker, permitindo observar o atraso associado à comunicação. O jitter (J) é calculado a partir da variação entre latências consecutivas e permite analisar a regularidade temporal do canal de comunicação.

A taxa de entrega de pacotes (PDR) corresponde à razão entre pacotes confirmados e pacotes enviados, indicando a eficiência da transmissão. O RSSI, por sua vez, representa a intensidade do sinal WiFi recebido pelo dispositivo, sendo utilizado como indicador das condições de conectividade durante a operação. Além dessas métricas, o tempo de failover (T_f) é considerado para medir o intervalo entre a detecção de falha no servidor primário e a conexão ao servidor secundário, enquanto o tempo de recuperação (T_r) expressa o intervalo necessário para o retorno do dispositivo ao servidor primário após o restabelecimento do serviço.

Também são observados o MTBF (*Mean Time Between Failures*), que representa o tempo médio entre falhas consecutivas do servidor primário, e o MTTR (*Mean Time To Repair*), que corresponde ao tempo médio necessário para reparo. Em conjunto, essas métricas fornecem uma base para analisar o comportamento da arquitetura quanto ao desempenho da comunicação, à continuidade do serviço e à resposta do sistema diante de falhas.

4. Metodologia

Esta seção descreve a metodologia adotada para avaliar o desempenho e a confiabilidade da arquitetura IoT proposta em um ambiente com redundância entre provedores de nuvem. O experimento foi conduzido em um testbed composto por um dispositivo embarcado, duas instâncias em nuvem configuradas de forma redundante e serviços de apoio para armazenamento, sincronização, monitoramento e injeção de falhas. A Figura **??** apresenta o fluxo metodológico descrito.

A metodologia foi organizada em etapas sequenciais. Na primeira etapa, foi realizada a configuração do ambiente experimental. Para isso, utilizou-se um microcontrolador ESP32 acoplado a um sensor DHT22 para geração e envio contínuo de dados. A infraestrutura em nuvem foi composta por duas máquinas virtuais, uma hospedada na Google Cloud Platform (GCP) e outra na Amazon Web Services (AWS), ambas configuradas com broker MQTT Mosquitto, banco de dados InfluxDB e Grafana. A comunicação entre o dispositivo e os servidores ocorreu por WiFi, com uso do protocolo MQTT.

Na etapa seguinte, foi realizada a coleta e publicação dos dados. Durante a execução, o ESP32 efetua a leitura contínua das informações e publica as mensagens no servidor ativo. A lógica embarcada inclui um mecanismo de failover automático, no qual, após falhas consecutivas de conexão ou publicação, o dispositivo passa a utilizar o servidor secundário. Periodicamente, o sistema verifica a disponibilidade do servidor primário, permitindo o retorno da comunicação quando sua operação é restabelecida.

Em seguida, foi realizada a persistência e sincronização dos dados. As mensagens publicadas são recebidas e armazenadas localmente em cada servidor por um serviço responsável por encaminhar os dados do broker MQTT para o InfluxDB. Além disso, foi adotado um mecanismo de sincronização entre réplicas, executado periodicamente, com o objetivo de identificar lacunas temporais e replicar dados ausentes entre os servidores.

Na etapa de execução dos cenários experimentais, o experimento foi dividido em três fases. A Fase 1 corresponde ao período inicial de operação em condições normais, utilizado como baseline para as métricas de desempenho e comunicação. A Fase 2 corresponde ao intervalo em que o mecanismo de injeção de falhas foi ativado no broker MQTT do servidor primário, permitindo avaliar o comportamento da arquitetura sob interrupções controladas. A Fase 3 corresponde ao período posterior à desativação da injeção de falhas, utilizado para observar o comportamento do sistema após o restabelecimento da operação normal. No experimento realizado, as durações dessas fases foram, respectivamente, 3,2 horas, 4,0 horas e 15,3 horas.

Durante a Fase 2, o mecanismo de injeção de falhas alternou períodos de funcionamento e indisponibilidade no broker MQTT do servidor primário, com tempos gerados a partir de distribuição exponencial. Após essa execução, foram realizados o monitoramento e a coleta das métricas. Caso os dados obtidos nessa etapa não sejam suficientes para a análise, o fluxo pode retornar à execução dos cenários experimentais, permitindo repetir os testes, ajustar parâmetros do ambiente ou ampliar o período de observação.

Na etapa de monitoramento e coleta das métricas, foram registrados indicadores relacionados ao desempenho e à confiabilidade da arquitetura, incluindo latência, jitter, disponibilidade, taxa de entrega de pacotes, perdas, duplicações, tempo de failover e tempo de recuperação. Além dos registros produzidos pelo dispositivo embarcado, foi utilizado um monitor externo para verificar periodicamente a acessibilidade dos brokers MQTT e registrar eventos de falha e recuperação. Por fim, os dados coletados foram organizados e analisados de forma comparativa, considerando tanto o comportamento da comunicação quanto a resposta do sistema diante das falhas induzidas.

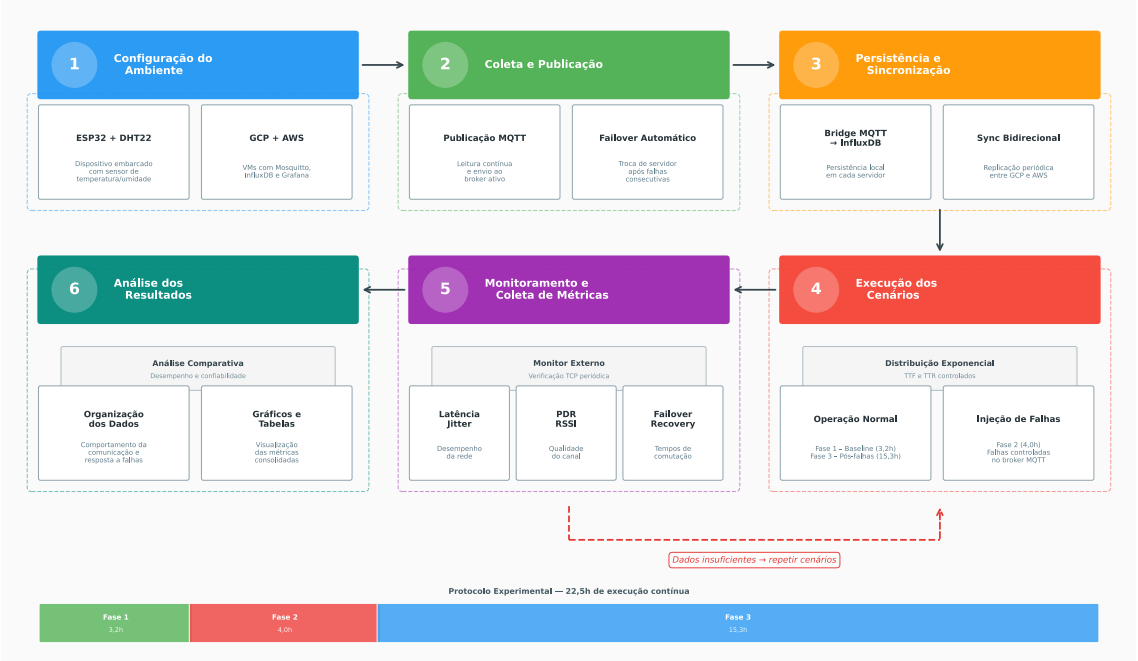

Figura 1. Fluxo metodológico de experimento

5. Arquitetura Proposta

A arquitetura proposta é composta por um nó de sensoriamento e por duas instâncias em nuvem, organizadas para permitir a continuidade da comunicação e a replicação de dados entre provedores distintos. A Figura 2 apresenta a visão geral do sistema. No lado do sensoriamento, um ESP32 acoplado a um sensor DHT22 realiza a coleta de dados e envia as medições por WiFi. No lado da nuvem, a infraestrutura é distribuída entre a Google Cloud Platform (GCP) e a Amazon Web Services (AWS), com cada instância executando Mosquitto, InfluxDB, Grafana e um serviço de bridge.

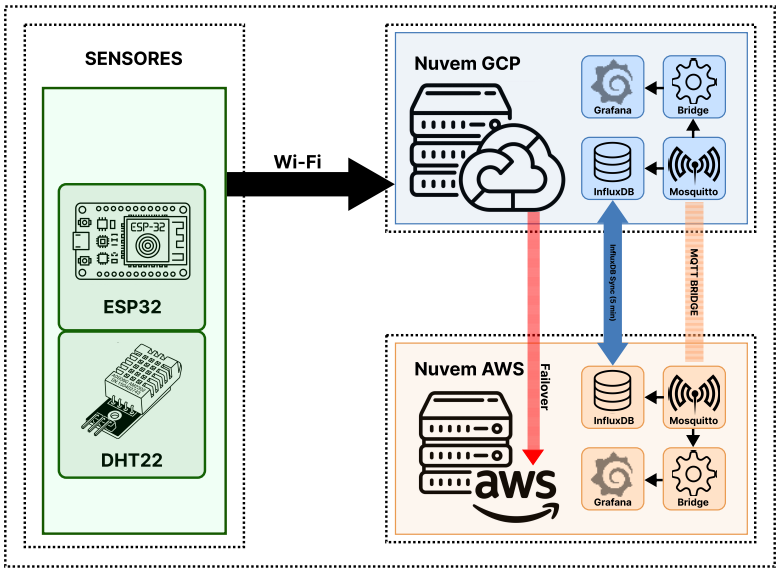

Figura 2. Arquitetura do sistema multi-cloud com failover e replicação.

No dispositivo de borda, o firmware realiza leituras periódicas do DHT22 e publica os dados por MQTT. Além das variáveis ambientais, o payload inclui informações de conectividade e operação, permitindo acompanhar o comportamento do sistema durante a execução. A comunicação ocorre por WiFi, com o ESP32 mantendo uma lista ordenada de servidores, em que o GCP é tratado como servidor primário e a AWS como servidor secundário. Quando a publicação ou a conexão falha de forma consecutiva, o dispositivo passa a utilizar o próximo servidor disponível da lista. Após operar no servidor secundário, o firmware verifica periodicamente a possibilidade de retorno ao primário. O Algoritmo 1 resume esse mecanismo de publicação com failover automático.

Nas instâncias em nuvem, os dados recebidos pelo broker Mosquitto são encaminhados ao InfluxDB, enquanto o Grafana consulta o banco para visualização em tempo real. A figura também mostra dois mecanismos de sincronização entre os provedores. O primeiro é uma bridge MQTT entre os brokers, permitindo o encaminhamento de mensagens entre GCP e AWS. O segundo é a sincronização entre os bancos InfluxDB, executada periodicamente, com o objetivo de reduzir lacunas de dados após interrupções ou comutações entre servidores. Dessa forma, quando um dos provedores se torna indisponível, a publicação continua no outro servidor e os dados podem ser sincronizados posteriormente entre as duas instâncias.

Algoritmo 1: Publicação MQTT com failover automático

```
Entrada: Servidores S = {GCP, AWS}, limiar de falhas k = 3
Saída: Dados publicados no broker ativo
i ← 0;
falhas ← 0;
enquanto verdadeiro faça
    dados ← LerSensores();
    payload ← FormatarPayload(dados);
    se MQTT.Publicar(S[i], payload) então
        falhas ← 0;
    senão
        falhas ← falhas + 1;
        se falhas ≥ k então
            i ← (i + 1) mod |S|;
            falhas ← 0;
        fim
    fim
    se i ≠ 0 e TempoDeVerificacao() então
        se TestarTCP(S[0]) então
            i ← 0;
        fim
    fim
fim
```

A utilização de dois provedores distintos reduz a dependência de uma única infraestrutura e distribui a operação entre ambientes independentes. Na configuração adotada, o GCP opera como instância principal e a AWS como instância de apoio, ambas com a mesma organização de serviços. Essa disposição permite acompanhar o comportamento da publicação, da comutação entre servidores e da replicação de dados em condições normais e em situações de falha.

Para a avaliação da arquitetura em condições de instabilidade controlada, foi incorporado ao ambiente um mecanismo de injeção de falhas no broker Mosquitto do servidor GCP. Esse mecanismo alterna períodos de operação e de interrupção do serviço, simulando indisponibilidades temporárias no servidor primário. Com isso, torna-se possível observar a resposta do sistema quanto à continuidade da publicação, ao redirecionamento do tráfego e ao retorno ao servidor principal. O Algoritmo 2 resume esse procedimento.

Algoritmo 2: Injeção de falhas no broker MQTT

```
Entrada: T_max: duração do experimento, μ_TTF, μ_TTR
Saída: Sequência de eventos de falha e reparo
t ← 0;
enquanto t < T_max faça
    TTF ← Exp(μ_TTF);
    Aguardar(TTF);
    PararMosquitto();
    TTR ← Exp(μ_TTR);
    Aguardar(TTR);
    IniciarMosquitto();
    t ← t + TTF + TTR;
fim
```

Ao longo da execução, o fluxo de dados segue um encadeamento simples. O ESP32 realiza a leitura dos sensores, formata o payload e publica a mensagem no servidor ativo. O broker Mosquitto recebe a publicação e a encaminha ao banco de dados local. Em paralelo, a bridge MQTT e a sincronização entre os bancos contribuem para manter os dados disponíveis nas duas instâncias. Quando o servidor primário se torna indis-

ponível, o failover transfere a publicação para o servidor secundário. Após a recuperação do GCP, o firmware pode restabelecer a comunicação com o servidor principal, mantendo a operação do sistema nas diferentes fases do experimento.

6. Resultados e Discussão

Esta seção apresenta os resultados obtidos ao longo de 22,5 horas de execução contínua do experimento. A análise foi organizada em quatro partes: métricas de rede durante a injeção de falhas, confiabilidade do sistema e condições ambientais, caracterização das falhas injetadas e comparação entre a arquitetura multi-cloud e o cenário com servidor único.

6.1. Métricas de Rede

A Figura 3 apresenta as métricas de rede obtidas durante o experimento. Na Figura 3(a), a disponibilidade do sistema é mostrada ao longo da fase com injeção de falhas, em intervalos de 10 minutos. Na configuração multi-cloud, os valores permaneceram próximos de 100% durante todo o período. No cenário com dependência exclusiva do servidor GCP, a disponibilidade foi de 26,37%, refletindo as interrupções no servidor primário. A Figura 3(b) mostra a latência média de publicação MQTT nas três fases do experimento. Na Fase 1, a média foi de 73,69 ms. Na Fase 2, esse valor passou para 37,10 ms, e na Fase 3 ficou em 40,0 ms. Esse comportamento acompanha a redistribuição do tráfego durante a indisponibilidade do GCP, com maior encaminhamento das publicações para a AWS. Os percentis seguiram a mesma tendência, com redução do P95 de 432 ms para 233 ms e do P99 de 637 ms para 436 ms entre as Fases 1 e 2.

A taxa de entrega de pacotes, mostrada na Figura 3(c), permaneceu acima de 99,99% ao longo da fase com falhas. Esse resultado indica que a publicação foi mantida com pouca variação mesmo durante as interrupções do servidor primário. A Figura 3(d) mostra a distribuição do tráfego entre GCP e AWS. Durante a Fase 2, houve redução gradual da participação do GCP e aumento correspondente da AWS. Enquanto o GCP concentrava 83,2% do tráfego na fase baseline, sua participação caiu para 21,2% durante a injeção de falhas. No mesmo intervalo, a AWS passou a responder por 78,8% das publicações. As oscilações ao longo do tempo acompanham os ciclos de falha e recuperação do servidor primário.

6.2. Confiabilidade e Dados Ambientais

A Figura 4 reúne informações sobre o comportamento dos servidores durante a fase com injeção de falhas, além das condições de conectividade e das variáveis ambientais registradas ao longo do experimento. A Figura 4(a) mostra o estado dos servidores GCP e AWS com base nos registros do monitor de eventos. Durante a Fase 2, o GCP alternou entre períodos de operação e falha, conforme definido no cenário experimental. A AWS permaneceu disponível na maior parte do tempo, com um intervalo entre aproximadamente 156 e 170 minutos em que ocorreram falhas breves. Nesse período, houve sobreposição entre indisponibilidades dos dois servidores.

A Figura 4(b) mostra o jitter médio e o percentil 95 calculados em janelas de 10 minutos durante a fase com falhas. O jitter médio permaneceu em uma faixa relativamente estável ao longo de boa parte do período, com elevações nos intervalos em que ocorreram

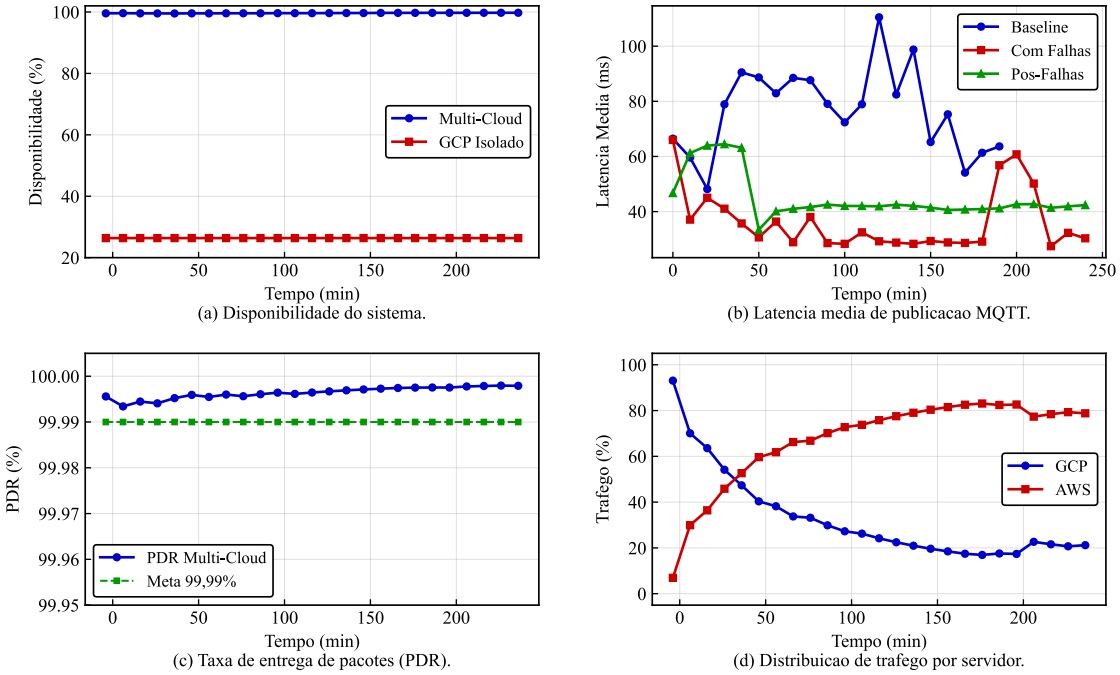

Figura 3. Métricas de rede

mudanças de servidor. O mesmo comportamento foi registrado no P95, indicando maior variação temporal da latência nos momentos de transição e recuperação da comunicação. A Figura 4(c) mostra a qualidade do sinal WiFi ao longo das 22,5 horas de experimento. O RSSI médio permaneceu em faixa estável, sem variações acentuadas entre os valores mínimo e máximo em cada janela de observação. Esse comportamento indica estabilidade da conexão sem fio do ESP32 durante a execução. A Figura 4(d) apresenta os valores de temperatura e umidade medidos pelo DHT22. As medições oscilaram ao longo do experimento dentro da faixa registrada pelo dispositivo, sem associação direta com as variações identificadas nas métricas de comunicação e confiabilidade.

6.3. Análise das Falhas Injetadas

Durante a Fase 2, o injetor de falhas produziu 9 ciclos de interrupção e reparo no broker Mosquitto do servidor GCP ao longo de aproximadamente 241 minutos de monitoramento. A Figura 5 reúne a duração das falhas registradas pelo monitor de eventos externo. No GCP, as falhas variaram de 2,2 minutos, no ciclo F1, a 43,2 minutos, no ciclo F7, com média de 19,8 minutos. Essa variação é compatível com a estratégia adotada para geração dos períodos de indisponibilidade e reparo. Os tempos registrados pelo monitor incluem tanto a interrupção do serviço quanto o intervalo até a detecção da falha e da posterior restauração, o que contribui para diferenças entre os tempos configurados e os tempos efetivamente medidos.

Ao longo dessa fase, o GCP permaneceu indisponível por aproximadamente 177,8 minutos, resultando em disponibilidade de 26,37%. Nesse intervalo, o ESP32 passou a utilizar com maior frequência o servidor secundário, totalizando 17 eventos de failover. Na AWS, foram registradas 7 falhas breves concentradas em um intervalo de cerca de 10 minutos. Como o mecanismo de injeção atuava apenas sobre o GCP, essas ocorrências não

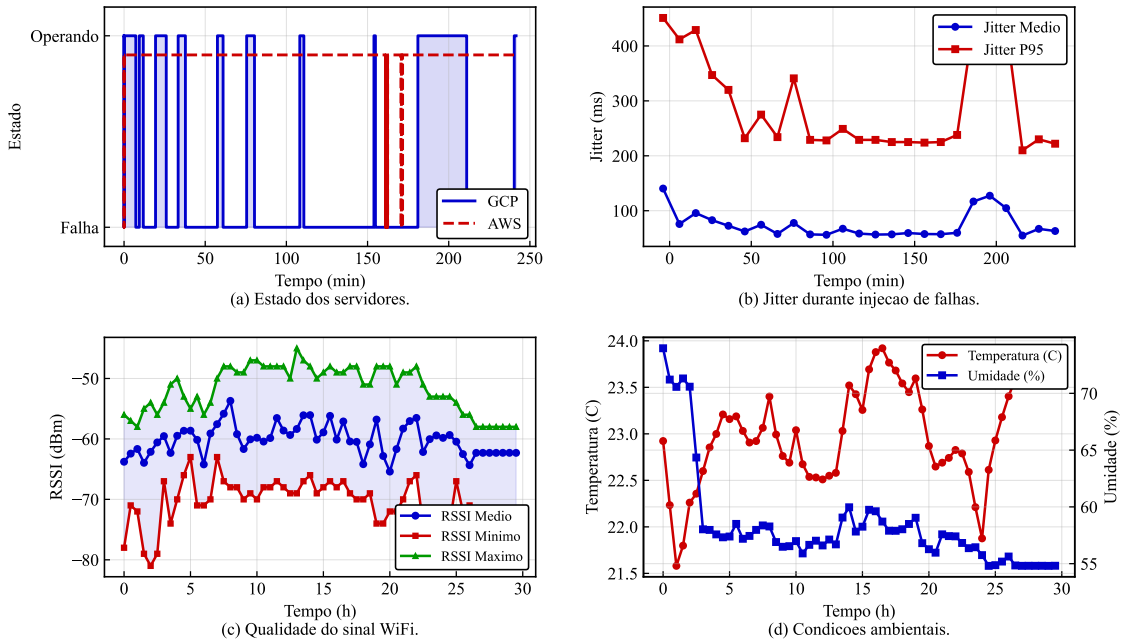

Figura 4. Confiabilidade e ambiente: (a) estado dos servidores, (b) jitter médio e P95, (c) RSSI do sinal WiFi e (d) temperatura e umidade.

foram produzidas diretamente pelo injetor. Uma possível explicação é a presença de instabilidades temporárias associadas ao aumento de reconexões e tentativas de publicação nesse período. As falhas variaram entre 5,1 s e 21,0 s, com média de 10,6 s. O tempo total de indisponibilidade foi de 74,4 s, resultando em disponibilidade de 99,49% na Fase 2.

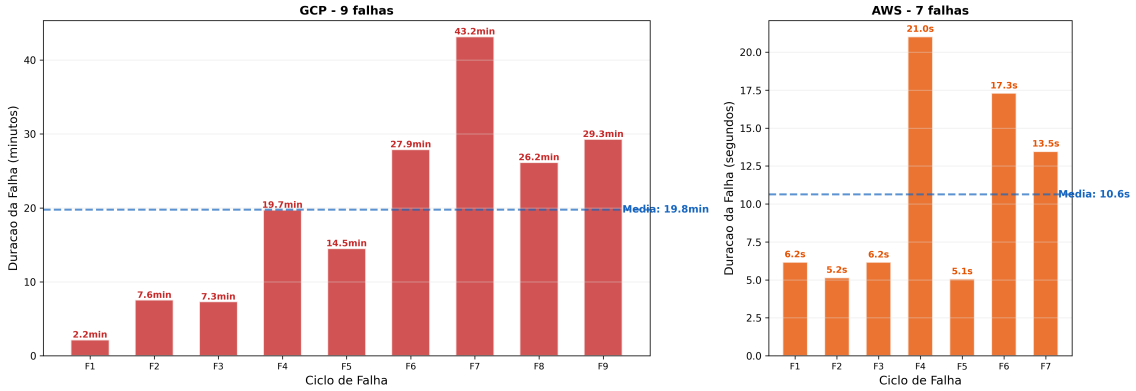

Figura 5. Duração das falhas registradas durante a Fase 2.

A Figura 6 mostra a evolução da disponibilidade acumulada dos servidores ao longo da Fase 2. No GCP, a curva apresenta reduções sucessivas ao longo dos ciclos de falha, estabilizando-se em torno de 26% ao final do período. Na AWS, os valores permaneceram próximos de 100%, com pequena redução no intervalo em que ocorreram as falhas breves.

A Tabela 2 reúne os indicadores da Fase 2, incluindo número de falhas, tempo total de indisponibilidade, a disponibilidade e duração média, mínima e máxima das falhas em cada servidor.

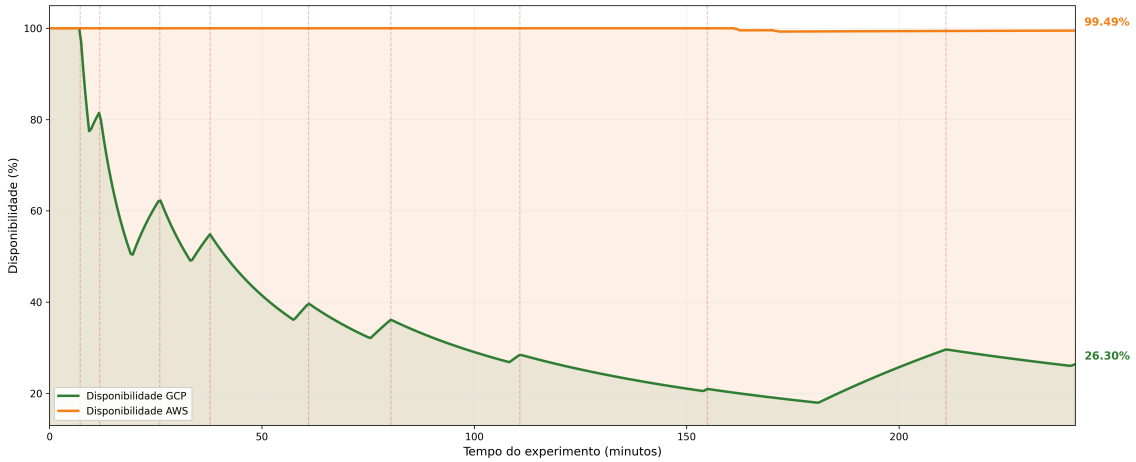

Figura 6. Disponibilidade acumulada dos servidores durante a Fase 2.

Tabela 2. Resumo da injeção de falhas na Fase 2.

Métrica	GCP (Primário)	AWS (Backup)
Duração total	242 min	242 min
Falhas registradas	9	7
Tempo em falha	177,8 min	1,2 min
Tempo operando	63,7 min	240,3 min
Disponibilidade (%)	26,37	99,49
Duração média de falha	19,8 min	10,6 s
Maior falha	43,2 min	21,0 s
Menor falha	2,2 min	5,1 s

6.4. Comparativo de Desempenho

A Figura 7 reúne o comparativo entre a configuração multi-cloud, com failover e replicação, e o cenário de operação com GCP isolado ao longo das três fases do experimento.

A Figura 7(a) mostra a disponibilidade em cada fase. Na Fase 1, as duas configurações apresentaram 99,23%, pois não houve interrupções relevantes no servidor primário. Na Fase 2, a configuração multi-cloud registrou 99,65%, enquanto o cenário com GCP isolado apresentou 26,37%. Na Fase 3, os dois cenários voltaram a apresentar valores próximos, com 99,87%. A Figura 7(b) mostra a latência média de publicação por fase. Na Fase 1, a latência média foi de 73,69 ms. Na Fase 2, esse valor passou para 37,10 ms. Na Fase 3, a latência média ficou em 40,0 ms. Os percentis P95 e P99 seguiram a mesma tendência, com valores de 432 ms e 637 ms na Fase 1, 233 ms e 436 ms na Fase 2, e 257 ms e 450 ms na Fase 3.

A Figura 7(c) mostra a distribuição do tráfego entre os servidores GCP e AWS. Na Fase 1, o GCP concentrou 83,2% das publicações, enquanto a AWS recebeu 16,8%. Na Fase 2, essa distribuição passou para 21,2% no GCP e 78,8% na AWS. Na Fase 3, o tráfego voltou a se concentrar majoritariamente no GCP, com 78,4%, enquanto a AWS passou a receber 21,6%. A Figura 7(d) mostra a quantidade de failovers e a taxa de entrega de pacotes em cada fase. A Fase 2 concentrou 17 failovers, enquanto a Fase 1 registrou 2 e a Fase 3 registrou 3. Em todas as fases, o PDR permaneceu acima de 99,93%, com valores de 99,99% na Fase 1, 99,99% na Fase 2 e 99,93% na Fase 3. A Tabela 3 resume

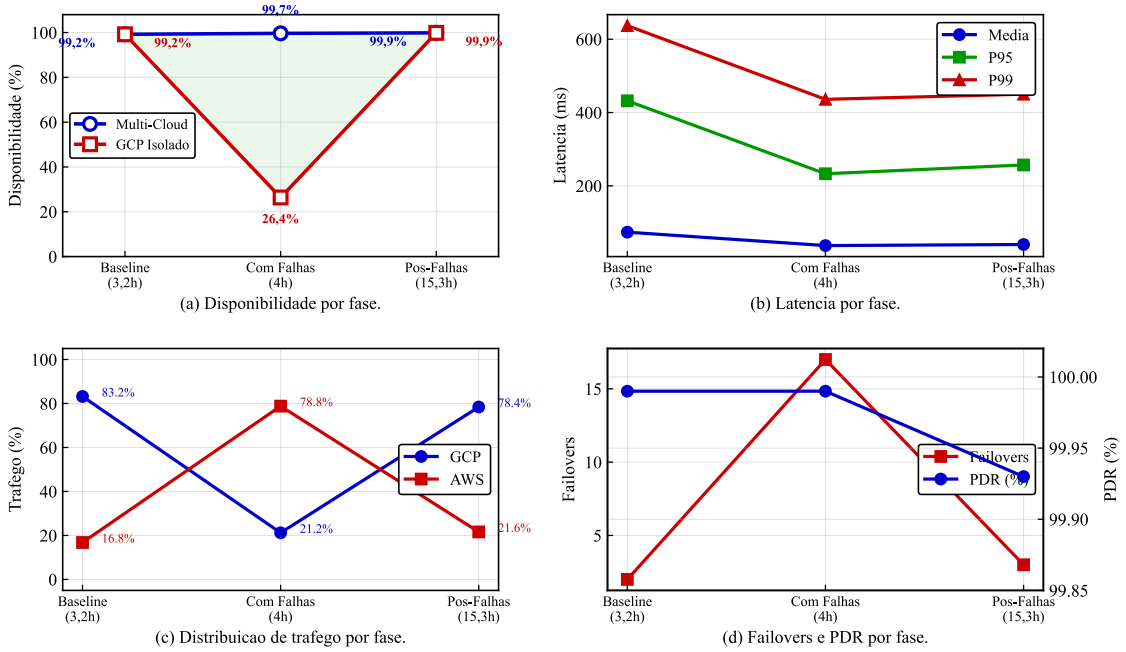

Figura 7. Comparativo por fase: disponibilidade, latência, distribuição de tráfego, failovers e PDR.

os resultados observados em cada fase do experimento.

Tabela 3. Resumo das métricas por fase do experimento.

Métrica	Fase 1 (Baseline)	Fase 2 (Falhas)	Fase 3 (Pós-falhas)
Duração (h)	3,2	4,0	15,3
Disponibilidade multi-cloud (%)	99,23	99,65	99,87
Disponibilidade GCP isolado (%)	99,23	26,37	99,87
Latência média (ms)	73,69	37,10	40,00
Latência P95 (ms)	432	233	257
Latência P99 (ms)	637	436	450
PDR (%)	99,99	99,99	99,93
Tráfego GCP (%)	83,2	21,2	78,4
Tráfego AWS (%)	16,8	78,8	21,6
Failovers	2	17	3

7. Conclusão

Este artigo apresentou uma avaliação experimental do desempenho e da confiabilidade de uma arquitetura IoT aplicada ao contexto médico, baseada em multi-cloud, com failover automático e replicação de dados entre provedores de nuvem.

Os resultados mostraram que a arquitetura manteve disponibilidade superior a 99,6% durante a fase com falhas, enquanto o cenário com servidor primário isolado apresentou 26,37% no mesmo período. A taxa de entrega de pacotes permaneceu acima de 99,93% em todas as fases, e as réplicas do InfluxDB mantiveram o mesmo conjunto de dados ao final do experimento. Além disso, a redistribuição do tráfego entre GCP e AWS acompanhou os ciclos de falha e recuperação.

Em conjunto, os resultados mostram que a combinação entre failover automático, replicação de dados e monitoramento contínuo permitiu manter a continuidade da publicação mesmo diante de falhas prolongadas no servidor primário. Para trabalhos futuros, será ampliada a avaliação para outros protocolos e tecnologias de comunicação, incluindo LoRaWAN, além da consideração de diferentes perfis de carga, novos cenários de falha e variações nas políticas de failover e recuperação.

Referências

Cao, R., Tang, Z., Liu, C., and Veeravalli, B. (2019). A scalable multicloud storage architecture for cloud-supported medical internet of things. *IEEE Internet of Things Journal*, 7(3):1641–1654.

Chen, Y. and Kunz, T. (2016). Performance evaluation of iot protocols under a constrained wireless access network. In *2016 International Conference on Selected Topics in Mobile & Wireless Networking (MoWNeT)*, pages 1–7. IEEE.

Lakhan, A., Lateef, A. A. A., Abd Ghani, M. K., Abdulkareem, K. H., Mohammed, M. A., Nedoma, J., Martinek, R., and Garcia-Zapirain, B. (2023). Secure-fault-tolerant efficient industrial internet of healthcare things framework based on digital twin federated fog-cloud networks. *Journal of King Saud University-Computer and Information Sciences*, 35(9):101747.

Neto, J. P. A., Pianto, D. M., and Ralha, C. G. (2019). Mults: A multi-cloud fault-tolerant architecture to manage transient servers in cloud computing. *Journal of Systems Architecture*, 101:101651.

Rahmani, A.-M., Thanigaivelan, N. K., Gia, T. N., Granados, J., Negash, B., Liljeberg, P., and Tenhunen, H. (2015). Smart e-health gateway: Bringing intelligence to internet-of-things based ubiquitous healthcare systems. In *2015 12th annual IEEE consumer communications and networking conference (CCNC)*, pages 826–834. IEEE.

Sornlertlamvanich, P., Phakaket, C., Hantula, P., Kanjanawattana, S., Kaoungku, N., and Srivisut, K. (2025). Integration of cloud-based central telemedicine system and iot device networks. *Computers*, 14(9):357.

Stergiou, C. L., Koidou, M. P., and Psannis, K. E. (2023). Iot-based big data secure transmission and management over cloud system: A healthcare digital twin scenario. *Applied Sciences*, 13(16):9165.

Upadhyay, S., Kumar, M., Upadhyay, A., Verma, S., Kavita, Kaur, M., Khurma, R. A., and Castillo, P. A. (2023). Challenges and limitation analysis of an iot-dependent system for deployment in smart healthcare using communication standards features. *Sensors*, 23(11):5155.

Wadullah Tareq, R. and Ahmed Khaleel, T. (2021). Implementation of mqtt protocol in health care based on iot systems: a study. *International journal of electrical and computer engineering systems*, 12(4):215–223.